

LAPORAN PRAKTIKUM W4: INHERITANCE, GENERALIZATION & OVERRIDING

Diajukan untuk memenuhi salah satu tugas praktikum mata kuliah Pemrograman Berorientasi Objek.



Disusun Oleh:
Rainissa Azizah (251511059)
Jurusan Teknik Komputer dan Informatika

Program Studi D-3 Teknik Informatika
Politeknik Negeri Bandung
2026

A. Exercise 1: The Circle and Cylinder Classes

1. Task 1.1: Modify class Circle

Class Circle dilengkapi sesuai class diagram, yaitu dengan menambah constructor Circle(radius, color), getColor(), setColor(), dan setRadius(). Format toString() disamakan dengan diagram menjadi Circle[radius=r,color=c]. Variabel radius dan color bersifat private, sehingga hanya bisa diakses lewat constructor, getter, dan setter.

```
public class Circle {
    private double radius;
    private String color;

    public Circle() {
        radius = 1.0;
        color = "red";
    }
    public Circle(double r) {
        radius = r;
        color = "red";
    }
    public Circle(double r, String c) {
        radius = r;
        color = c;
    }
    public double getRadius() {
        return radius;
    }
    public void setRadius(double r) {
        radius = r;
    }
    public String getColor() {
        return color;
    }
    public void setColor(String c) {
        color = c;
    }
    public double getArea() {
        return radius * radius * Math.PI;
    }
    public String toString() {
        return "Circle[radius=" + radius + " color=" + color + "];";
    }
}
```

```
}
```

2. Task 1.2: Overriding the getArea() method

Class Cylinder dibuat sebagai turunan Circle memakai extends, dan constructor-nya memanggil constructor Circle lewat super(). Method getArea() di-override untuk menghitung luas permukaan, dengan luas alas diambil dari super.getArea(). Setelah itu, getVolume() ikut memakai super.getArea() supaya volume tetap dihitung dari luas alas, bukan luas permukaan.

```
public class Cylinder extends Circle {
    private double height;

    public Cylinder(){
        super();
        height = 1.0;
    }
    public Cylinder(double height) {
        super();
        this.height = height;
    }
    public Cylinder(double radius, double height) {
        super(radius);
        this.height = height;
    }
    public Cylinder(double radius, double height, String color) {
        super(radius, color);
        this.height = height;
    }
    public double getHeight() {
        return height;
    }
    public void setHeight(double h) {
        height = h;
    }
    public double getVolume() {
        return super.getArea()*height;
    }
    @Override
    public double getArea() {
        return 2 * Math.PI * getRadius() * height + 2 * super.getArea();
    }
    @Override
    public String toString() {
```

```

        return "Cylinder: subclass of " + super.toString() + " height=" + height;
    }
}

```

3. Task 1.3: Provide a toString() method

Pada task ini, method toString() ditambahkan di class Cylinder. Method ini meng-override toString() milik Circle, tetapi tetap memanggil super.toString() supaya data radius dan warna tampil dengan format dari Circle. Setelah itu, ditambahkan keterangan bahwa object tersebut adalah subclass beserta nilai tingginya. Anotasi @Override juga ditulis di atas method ini. Fungsinya untuk meminta compiler memeriksa bahwa method tersebut memang ada di superclass, sehingga kalau nama method salah ketik, kesalahan langsung terlihat lewat pesan error dan tidak sampai membuat method baru tanpa disadari.

4. Hasil program TestCylinder

```

PS C:\Users\User\OneDrive - Politeknik Negeri Bandung\Kuliah\Semester 3\PBO\PBO Praktik\Praktikum PBO W4\Exercise1>
PS C:\Users\User\OneDrive - Politeknik Negeri Bandung\Kuliah\Semester 3\PBO\PBO Praktik\Praktikum PBO W4\Exercise1>
java Circle.java Cylinder.java TestCylinder.java
PS C:\Users\User\OneDrive - Politeknik Negeri Bandung\Kuliah\Semester 3\PBO\PBO Praktik\Praktikum PBO W4\Exercise1>
java TestCylinder
Cylinder: radius=1.0 height=1.0 base area=12.566370614359172 volume=3.141592653589793
Cylinder: radius=1.0 height=10.0 base area=69.11503837897544 volume=31.41592653589793
Cylinder: radius=2.0 height=10.0 base area=150.79644737231007 volume=125.66370614359172
Cylinder: subclass of Circle[radius=1.0 color=red] height=1.0
Cylinder: subclass of Circle[radius=2.0 color=red] height=10.0
PS C:\Users\User\OneDrive - Politeknik Negeri Bandung\Kuliah\Semester 3\PBO\PBO Praktik\Praktikum PBO W4\Exercise1>

```

B. Exercise 2: Superclass Shape and its Subclasses Circle, Rectangle and Square

1. Class Shape

Pada task ini dibuat class Shape sebagai superclass yang menyimpan data umum dari sebuah bentuk, yaitu warna (color) dan status terisi atau tidak (filled). Class ini punya dua constructor. Constructor tanpa parameter mengisi nilai default, yaitu warna "red" dan filled bernilai true, sedangkan constructor lainnya menerima warna dan filled sesuai yang diinginkan. Kedua variabel dibuat private, sehingga diakses lewat getter dan setter. Khusus variabel bertipe boolean, getter-nya diberi nama isFilled(), bukan getFilled(), sesuai aturan penamaan di Java. Method toString() menghasilkan teks dengan format Shape[color=?,filled=?].

```

public class Shape {
    private String color;
    private boolean filled;

    // Shape()
    public Shape() {
        color = "red";
        filled = true;
    }
}

```

```

    }
    // Shape(color:String, filled:boolean)
    public Shape(String c, boolean f) {
        color = c;
        filled = f;
    }
    // getColor():String
    public String getColor() {
        return color;
    }
    // setColor(color:String):void
    public void setColor(String c) {
        color = c;
    }
    // isFilled():boolean
    public boolean isFilled() {
        return filled;
    }
    // setFilled(filled:boolean):void
    public void setFilled(boolean f) {
        filled = f;
    }
    // toString():String
    public String toString() {
        return "Shape[color=" + color + ",filled=" + filled + "];";
    }
}

```

2. Class Circle sebagai subclass Shape

Class Circle dibuat sebagai turunan dari Shape memakai extends, sehingga warna dan status filled diwarisi dari Shape dan hanya radius yang perlu ditambahkan. Constructor Circle memanggil constructor Shape lewat super() untuk mengisi nilai default, atau lewat super(color, filled) jika warna dan filled ditentukan sendiri. Pemanggilan ini diperlukan karena color dan filled bersifat private di Shape, sehingga tidak bisa diisi langsung dari Circle. Selain itu, Circle punya method getArea() untuk menghitung luas ($\pi \times \text{radius}^2$) dan getPerimeter() untuk menghitung keliling ($2 \times \pi \times \text{radius}$). Method toString() di-override dengan memanggil super.toString(), lalu ditambah nilai radius, sehingga hasilnya berbentuk Circle[Shape[color=?,filled=?],radius=?].

```

public class Circle extends Shape {
    private double radius;

```

```

// Circle()
public Circle() {
    super();
    radius = 1.0;
}
// Circle(radius:double)
public Circle(double r) {
    super();
    radius = r;
}
// Circle(radius:double, color:String, filled:boolean)
public Circle(double r, String c, boolean f) {
    super(c, f);
    radius = r;
}
// getRadius():double
public double getRadius() {
    return radius;
}
// setRadius(radius:double):void
public void setRadius(double r) {
    radius = r;
}
// getArea():double
public double getArea() {
    return radius * radius * Math.PI;
}
// getPerimeter():double
public double getPerimeter() {
    return 2 * Math.PI * radius;
}
// toString():String
@Override
public String toString() {
    return "Circle[" + super.toString() + ",radius=" + radius + "]";
}
}

```

3. Class Rectangle sebagai subclass Shape

Class Rectangle juga turunan dari Shape dan menambahkan dua variabel, yaitu width dan length, yang keduanya bernilai 1.0 pada constructor default. Seperti pada Circle, constructor Rectangle memakai super() atau super(color, filled) untuk mengisi data dari Shape. Luas dihitung dengan $\text{width} \times \text{length}$, sedangkan keliling dengan $2 \times (\text{width} + \text{length})$. Method toString() di-override dengan memanggil super.toString() dan menambahkan nilai width dan length.

```
public class Rectangle extends Shape {
    private double width;
    private double length;

    // Rectangle()
    public Rectangle() {
        super();
        width = 1.0;
        length = 1.0;
    }
    // Rectangle(width:double, length:double)
    public Rectangle(double w, double l) {
        super();
        width = w;
        length = l;
    }
    // Rectangle(width:double, length:double, color:String, filled:boolean)
    public Rectangle(double w, double l, String c, boolean f) {
        super(c, f);
        width = w;
        length = l;
    }
    // getWidth():double
    public double getWidth() {
        return width;
    }
    // setWidth(width:double):void
    public void setWidth(double w) {
        width = w;
    }
    // getLength():double
    public double getLength() {
        return length;
    }
}
```

```

    }
    // setLength(length:double):void
    public void setLength(double l) {
        length = l;
    }
    // getArea():double
    public double getArea() {
        return width * length;
    }
    // getPerimeter():double
    public double getPerimeter() {
        return 2 * (width + length);
    }
    // toString():String
    @Override
    public String toString() {
        return "Rectangle[" + super.toString() + ",width=" + width + ",length=" +
length + "]\n";
    }
}

```

4. Class Square sebagai subclass Rectangle

Persegi adalah persegi panjang dengan sisi yang sama, sehingga Square cocok menjadi turunan Rectangle. Square tidak punya variabel sendiri, dan constructor-nya memanggil `super(side, side)`. `getArea()` dan `getPerimeter()` tidak perlu di-override karena rumus dari Rectangle sudah benar. Yang di-override adalah `setWidth()` dan `setLength()`, supaya keduanya mengubah width dan length sekaligus dan bentuknya tetap persegi.

```

public class Square extends Rectangle {
    // Square()
    public Square() {
        super();
    }
    // Square(side:double)
    public Square(double side) {
        super(side, side);
    }
    // Square(side:double, color:String, filled:boolean)
    public Square(double side, String c, boolean f) {
        super(side, side, c, f);
    }
    // getSide():double

```

```

    public double getSide() {
        return getWidth();
    }
    // setSide(side:double):void
    public void setSide(double side) {
        super.setWidth(side);
        super.setLength(side);
    }
    // setWidth(side:double):void
    @Override
    public void setWidth(double side) {
        super.setWidth(side);
        super.setLength(side);
    }
    // setLength(side:double):void
    @Override
    public void setLength(double side) {
        super.setWidth(side);
        super.setLength(side);
    }
    // toString():String
    @Override
    public String toString() {
        return "Square[" + super.toString() + "]\n";
    }
}

```

5. Hasil program TestShape

```

PS C:\Users\User\OneDrive - Politeknik Negeri Bandung\Kuliah\Semester 3\PBO\PBO Praktikum\Praktikum PBO W4\Exercise2>
javac Shape.java Circle.java Rectangle.java Square.java TestShape.java
PS C:\Users\User\OneDrive - Politeknik Negeri Bandung\Kuliah\Semester 3\PBO\PBO Praktikum\Praktikum PBO W4\Exercise2>
java TestShape
Shape[color=red, filled=true]
Shape[color=blue, filled=false]
green false
Shape[color=green, filled=false]
Circle[Shape[color=red, filled=true], radius=1.0]
Circle[Shape[color=red, filled=true], radius=2.0]
Circle[Shape[color=blue, filled=false], radius=3.0]
radius=5.0 area=78.53981633974483 perimeter=31.41592653589793
Rectangle[Shape[color=red, filled=true], width=1.0, length=1.0]
Rectangle[Shape[color=red, filled=true], width=2.0, length=3.0]
Rectangle[Shape[color=yellow, filled=false], width=4.0, length=5.0]
width=2.0 length=3.0 area=6.0 perimeter=10.0
Square[Rectangle[Shape[color=red, filled=true], width=1.0, length=1.0]]
Square[Rectangle[Shape[color=red, filled=true], width=4.0, length=4.0]]
Square[Rectangle[Shape[color=green, filled=true], width=2.0, length=2.0]]
side=4.0 area=16.0 perimeter=16.0
Square[Rectangle[Shape[color=red, filled=true], width=6.0, length=6.0]]
Square[Rectangle[Shape[color=red, filled=true], width=7.0, length=7.0]]
Square[Rectangle[Shape[color=red, filled=true], width=3.0, length=3.0]]

```

C. Exercise 3: Multiple Inheritance

1. Class Employee dan EmployeeTest

Class Employee dipakai untuk menyimpan data seorang pegawai, yaitu nama, gaji, dan tanggal masuk kerja. Semua variabelnya bersifat private, sehingga data hanya bisa diakses lewat method. Method raiseSalary() menaikkan gaji sebesar persentase tertentu dengan cara mengalikan gaji dengan 1 + persen / 100, sedangkan hireYear() mengembalikan tahun masuk kerja dan print() mencetak nama, gaji, dan tahun masuk. Program EmployeeTest membuat sebuah array berisi tiga object Employee, menaikkan gaji ketiganya sebesar 5%, lalu mencetak datanya. Hasilnya, gaji masing-masing pegawai naik 5% dari gaji awal.

a. Kode Employee.java

```
class Employee {
    public Employee(String n, double s, int day, int month, int year) {
        name = n;
        salary = s;
        hireday = day;
        hiremonth = month;
        hireyear = year;
    }
    public void print() {
        System.out.println(name + " " + salary + " " + hireYear());
    }
    public void raiseSalary(double byPercent) {
        salary *= 1 + byPercent / 100;
    }
    public int hireYear() {
        return hireyear;
    }
    private String name;
    private double salary;
    private int hireday;
    private int hiremonth;
    private int hireyear;
}
```

b. Kode EmployeeTest.java

```
public class EmployeeTest {
    public static void main(String[] args) {
        Employee[] staff = new Employee[3];
        staff[0] = new Employee("Antonio Rossi", 2000000, 1, 10, 1989);
```

```

        staff[1] = new Employee("Maria Bianchi", 2500000, 1, 12, 1991);
        staff[2] = new Employee("Isabel Vidal", 3000000, 1, 11, 1993);

        int i;
        for (i = 0; i < 3; i++) staff[i].raiseSalary(5);
        for (i = 0; i < 3; i++) staff[i].print();
    }
}

```

c. Output EmployeeTest

```

PS C:\Users\User\OneDrive - Politeknik Negeri Bandung\Kuliah\Semester 3\
PBO\PBO Praktik\Praktikum PBO W4\Exercise3> java EmployeeTest
Antonio Rossi 2100000.0 1989
Maria Bianchi 2625000.0 1991
Isabel Vidal 3150000.0 1993

```

2. Class Manager dan ManagerTest

Class Manager dibuat sebagai turunan Employee memakai extends, sehingga data dan method pegawai bisa dipakai kembali tanpa ditulis ulang. Constructor Manager memanggil constructor Employee lewat super(), lalu menambahkan data khusus manager, yaitu secretaryName. Method raiseSalary() di-override karena manager mendapat bonus 0,5% untuk setiap tahun masa kerja, dihitung dari tahun sekarang dikurangi tahun masuk (diambil lewat hireYear()). Perlu ditambahkan import java.util.* agar GregorianCalendar dan Calendar dikenali.

a. Kode Manager.java

```

import java.util.*;

class Manager extends Employee {
    public Manager(String n, double s, int d, int m, int y) {
        super(n, s, d, m, y);
        secretaryName = "";
    }

    public void raiseSalary(double byPercent) {
        // tambah bonus 1/2% untuk setiap tahun masa kerja
        GregorianCalendar todaysDate = new GregorianCalendar();
        int currentYear = todaysDate.get(Calendar.YEAR);
        double bonus = 0.5 * (currentYear - hireYear());
        super.raiseSalary(byPercent + bonus);
    }

    public String getSecretaryName() {

```

```

        return secretaryName;
    }
    private String secretaryName;
}

```

b. Kode ManagerTest.java

```

public class ManagerTest {
    public static void main(String[] args) {
        Employee[] staff = new Employee[3];
        staff[0] = new Employee("Antonio Rossi", 2000000, 1, 10, 1989);
        staff[1] = new Manager("Maria Bianchi", 2500000, 1, 12, 1991);
        staff[2] = new Employee("Isabel Vidal", 3000000, 1, 11, 1993);

        int i;
        for (i = 0; i < 3; i++) staff[i].raiseSalary(5);
        for (i = 0; i < 3; i++) staff[i].print();
    }
}

```

c. Output ManagerTest

```

PS C:\Users\User\OneDrive - Politeknik Negeri Bandung\Kuliah\Semester 3\
PBO\PBO Praktik\Praktikum PBO W4\Exercise3> java ManagerTest
Antonio Rossi 2100000.0 1989
Maria Bianchi 3062500.0 1991
Isabel Vidal 3150000.0 1993

```

3. Case 1: Class abstract Sortable

Sortable adalah abstract class, sehingga tidak bisa dibuat object secara langsung dan hanya menjadi dasar bagi class lain. Di dalamnya ada method `compare()` yang bersifat abstract (tanpa isi, sehingga wajib diisi oleh class turunan) dan method static `shell_sort()` yang mengurutkan array dengan memanggil `compare()`. Agar bisa diurutkan, `Employee` diubah menjadi turunan `Sortable` dan mengisi `compare()` dengan membandingkan gaji, yaitu mengembalikan -1 jika lebih kecil, +1 jika lebih besar, dan 0 jika sama.

a. Kode Sortable.java

```

abstract class Sortable {
    public abstract int compare(Sortable b);
    public static void shell_sort(Sortable[] a) {
        int n = a.length;
        for (int gap = n / 2; gap > 0; gap /= 2) {
            for (int i = gap; i < n; i++) {
                Sortable temp = a[i];
                int j;
            }
        }
    }
}

```

```

        for (j = i; j >= gap && a[j - gap].compare(temp) > 0; j -= gap) {
            a[j] = a[j - gap];
        }
        a[j] = temp;
    }
}
}
}

```

b. Kode Employee.java yang baru

```

class Employee extends Sortable {
    public Employee(String n, double s, int day, int month, int year) {
        name = n;
        salary = s;
        hireday = day;
        hiremonth = month;
        hireyear = year;
    }
    public void print() {
        System.out.println(name + " " + salary + " " + hireYear());
    }
    public void raiseSalary(double byPercent) {
        salary *= 1 + byPercent / 100;
    }
    public int hireYear() {
        return hireyear;
    }
    public int compare(Sortable b) {
        Employee eb = (Employee) b;
        if (salary < eb.salary) return -1;
        if (salary > eb.salary) return +1;
        return 0;
    }
    private String name;
    private double salary;
    private int hireday;
    private int hiremonth;
    private int hireyear;
}

```

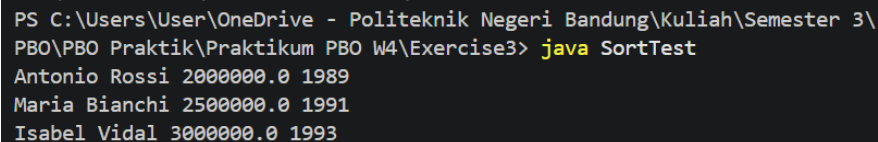
4. Case 2: Manager extends Employee extends Sortable

Menulis class Manager extends Employee extends Sortable tidak bisa dilakukan karena Java hanya mengizinkan satu extends per class. Namun, penulisan itu memang tidak diperlukan. Manager sudah turunan Employee dan Employee sudah turunan Sortable, sehingga Manager otomatis mewarisi compare() lewat Employee. Jadi Manager bisa diurutkan tanpa mengubah kodenya. Hal ini dibuktikan lewat SortTest, yaitu array berisi Employee dan Manager dengan gaji yang diacak, yang setelah Sortable.shell_sort() terurut dari gaji terkecil ke terbesar.

a. Kode SortTest.java

```
public class SortTest {  
    public static void main(String[] args) {  
        Employee[] staff = new Employee[3];  
        staff[0] = new Employee("Isabel Vidal", 3000000, 1, 11, 1993);  
        staff[1] = new Manager("Maria Bianchi", 2500000, 1, 12, 1991);  
        staff[2] = new Employee("Antonio Rossi", 2000000, 1, 10, 1989);  
  
        Sortable.shell_sort(staff);  
        for (int i = 0; i < 3; i++) staff[i].print();  
    }  
}
```

b. Output SortTest



```
PS C:\Users\User\OneDrive - Politeknik Negeri Bandung\Kuliah\Semester 3\  
PBO\PBO Praktik\Praktikum PBO W4\Exercise3> java SortTest  
Antonio Rossi 2000000.0 1989  
Maria Bianchi 2500000.0 1991  
Isabel Vidal 3000000.0 1993
```

D. Lesson Learn

Praktikum ini menunjukkan bahwa override suatu method bisa memengaruhi method lain yang memakainya. Contohnya getVolume() pada Cylinder yang menjadi salah setelah getArea() di-override dan baru benar setelah diganti dengan super.getArea(). Dari praktikum ini juga dipahami bahwa inheritance memungkinkan subclass memakai ulang variabel dan method milik superclass sehingga kode tidak perlu ditulis berulang. Constructor superclass dipanggil lewat super(), dan method superclass yang sudah di-override tetap bisa dipanggil lewat super.namaMethod(). Penulisan super ini penting, karena kalau tidak ditulis, method akan memanggil dirinya sendiri terus-menerus sampai program error.

Link Repo

<https://github.com/rainissa/PBO-Praktik.git>